

Отказоустойчивый веб-хостинг в VK Cloud

Итоговый проект

Кирилл Ремизов

23 июля 2026 г.

Цель проекта

Выбор варианта

Архитектура

Стек технологий

Terraform

CI/CD

Проверка

Результаты

Цель проекта

Развернуть **отказоустойчивую инфраструктуру** для веб-приложения (сайт-резюме) в Docker-контейнере на платформе **VK Cloud**.

Ключевые требования:

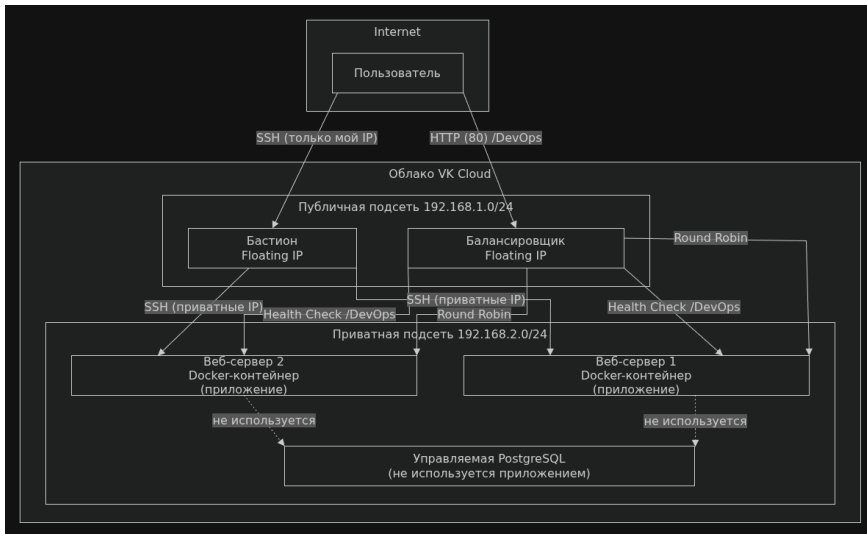
- Инфраструктура как код (Terraform)
- CI/CD через GitHub Actions
- Управляемая БД PostgreSQL
- Балансировка нагрузки (L7)
- Доступность при отказе одного веб-сервера (SLA 99.9%)

Вариант №1 — «Отказоустойчивый веб-хостинг на VM»
(с заменой ручного Nginx на Docker-контейнер).

Почему выбран:

- Покрывает все темы курса: сеть, безопасность, балансировка, БД, автоматизация
- Навыки Terraform, Docker, CI/CD, облачные сервисы VK Cloud
- Реальный промышленный стек
- Чёткие критерии успеха и наглядная демонстрация отказоустойчивости

Архитектура решения



Компоненты инфраструктуры

Виртуальные машины

- 2 веб-сервера (Basic-1-1-10) в приватной подсети
- Бастион (Basic-1-1-10) в публичной подсети

Балансировщик

- L7 Application Load Balancer
- Публичный Floating IP
- Health-check: GET /DevOps
- Round-robin

База данных

- PostgreSQL 14 (Standard-2-8-50)
- В приватной подсети
- Только изолированный доступ

Docker

- Flask + Gunicorn приложение
- Образ из приватного реестра
- -restart=always

VPC / Подсети

- `public` (192.168.1.0/24) — балансировщик + бастион
- `private` (192.168.2.0/24) — веб-серверы + БД
- Internet-шлюз через роутер

Security Groups

- `main`: HTTP (80) для всех, SSH (22) только с моего IP
- `db_sg`: PostgreSQL (5432) только из приватной подсети
- Доступ к веб-серверам — только через бастион

Единственная точка входа — бастион с Floating IP,
SSH разрешён только с IP автора

Компонент	Технология
Инфраструктура	Terraform (provider vkcs)
Оркестрация	Docker (контейнеры на BM)
Балансировка	VK Cloud Load Balancer (L7)
База данных	VK Cloud DBaaS (PostgreSQL 14)
CI/CD	GitHub Actions
Хранение состояния	S3 (VK Cloud Object Storage)
Приложение	Python (Flask) + Gunicorn

Rust, C++, Linux, Bash — доп. компетенции

Terraform — инфраструктура как код

Terraform-конфигурация разбита на 4 файла:

- `main.tf` — провайдеры, сеть, ВМ, балансировщик, БД
- `variables.tf` — все переменные с умолчаниями
- `outputs.tf` — выходные данные (IP, параметры БД)
- `backend.tf` — S3 backend для хранения state

Ключевые ресурсы

`vkcs_compute_instance.web[count=2]` — 2 веб-сервера
`vkcs_lb_loadbalancer` — L7 балансировщик
`vkcs_db_instance.postgres` — управляемая PostgreSQL
`vkcs_compute_instance.bastion` — бастион-хост

Terraform — user_data веб-сервера

```
#!/bin/bash
apt-get update
apt-get install -y docker.io
systemctl enable docker
systemctl start docker
sleep 5
docker run -d --restart=always \
  -p 80:80 ${var.docker_image}
```

- Автоматическая установка Docker при создании VM
- Pull из приватного реестра (до 5 повторных попыток)
- -restart=always — автоматический перезапуск

GitHub Actions — CI/CD Pipeline

Jobs:

1. **Plan** — инициализация, валидация, план
2. **Apply** — применение (только push в main)
3. **Destroy** — удаление инфраструктуры

Secrets:

- > VKCS_USERNAME
- > VKCS_PASSWORD
- > VKCS_PROJECT_ID
- > SSH_PUBLIC_KEY
- > S3_ACCESS/SECRET_KEY



Push в main → автоматический деплой инфраструктуры

Проверка работоспособности

1. Просмотр ресурсов в консоли VK Cloud
2. Доступность сайта через браузер
3. SSH через бастинг к веб-серверам, `docker ps`
4. **Отказоустойчивость:** `docker stop` на web-1
5. Запуск контейнера обратно, возврат в пул
6. Подключение к PostgreSQL через `psql`
7. Нагрузочное тестирование `ab`

<http://95.163.213.173/DevOps>

→ сайт продолжает работать

Результаты нагрузочного тестирования

```
ab -n 1000 -c 10 http://95.163.213.173/DevOps
```

Параметр	Значение
Полных запросов	1000
Ошибок	1 (Length)
Запросов/сек	76.54
Время на запрос (mean)	130.65 мс
Transfer rate	816.36 КБ/с
95% перцентиль	113 мс

99.9% успешных запросов — цель достигнута

Результаты проекта

Выполнено:

- Отказоустойчивая инфраструктура в VK Cloud
- Fully automated CI/CD (GitHub Actions)
- Infrastructure as Code (Terraform)
- Docker-контейнеризация приложения
- L7 балансировка + Health-check
- Управляемая PostgreSQL
- Нагрузочное тестирование

Задел на будущее:

- Пароль БД в коде (Vault)
- Health-check по /DevOps (не /health)
- Образ с тегом latest
- Нет мониторинга (Prometheus/Grafana)
- Нет автообновления контейнеров

Репозитории проекта

 GitHub (основной)

https://github.com/BlackCorbeau/final_assessment

Forgejo (зеркало)

https://git.rustprogersteam.ru/Black_Raven/MyResume

Спасибо за внимание!